STUDENT DATABASE MANAGEMENT SYSTEM

Algorithm and Dataset

1. STEP-BY-STEP PROCEDURE

System Initialization

1. Start the application and establish connection to the SQLite/MySQL database.
2. Check if the students table exists; if not, create it with required schema.
3. Display the main menu with options: Add / View / Search / Update / Delete / Exit.

Add Student Record

1. Accept student details: Name, Roll Number, Course, Branch, Year, Contact.
2. Validate all fields and check for duplicate Roll Number.
3. Execute INSERT INTO students query and commit the transaction.

View / Search Student Records

1. User selects View All or Search by Roll No / Name / Department.
2. Execute SELECT query with or without WHERE clause.
3. Display matching records; show 'No records found' if empty.

Update Student Record

1. Accept Roll Number of the student to update.
2. Verify record exists using SELECT query.
3. Accept new values and execute UPDATE students SET ... WHERE roll_number = ? query.

Delete Student Record

1. Accept Roll Number and verify record existence.
2. Display details and ask for user confirmation.
3. Execute DELETE FROM students WHERE roll_number = ? and commit.

Exit

1. Commit all pending transactions.
2. Close database connection securely and terminate the application.

2. LOGIC OR METHOD USED

The system uses a modular programming approach where each CRUD operation is a separate function. Core logic is built on SQL query execution via Python's sqlite3 / mysql-connector library.

- Validation Logic: All input fields are checked before executing any database query.
- Duplicate Prevention: Roll Number uniqueness is enforced via SELECT before INSERT.
- Exception Handling: try-except blocks manage runtime errors and DB failures.
- Transaction Management: commit() ensures data integrity; rollback() reverts on failure.
- Search Optimization: SQL WHERE clause with Roll Number enables fast lookups.

3. MODEL OR TECHNIQUE APPLIED

Programming Paradigm: Procedural / Modular Programming using Python 3.x

Database Model: Relational Database Model — SQLite (development) / MySQL (production)

Operations Framework: CRUD — Create, Read, Update, Delete via SQL queries

Database Schema — students Table:

Column Name	Data Type	Description
id	INTEGER (PK, AUTO)	Auto-generated unique student ID
name	VARCHAR(100)	Full name of the student
roll_number	VARCHAR(20) UNIQUE	Unique roll number identifier
course	VARCHAR(50)	Course enrolled (e.g., B.Tech)
branch	VARCHAR(50)	Branch/Department (e.g., CSE)
year	INTEGER	Current academic year (1–4)
contact	VARCHAR(15)	Student contact number

4. PROCESS FLOW OF THE SYSTEM

Step 01 [START] → Application launches and initializes database connection.

Step 02 [INIT DB] → Check/create students table in the database.

Step 03 [MAIN MENU] → Display options: Add / View / Search / Update / Delete / Exit.

Step 04 [USER INPUT] → Accept user's choice and validate the input.

Step 05 [DECISION] → Route to the appropriate module based on user selection.

Step 06 [EXECUTE SQL] → Run the corresponding SQL query (INSERT / SELECT / UPDATE / DELETE).

Step 07 [RESULT] → Display success message or fetched records to the user.

Step 08 [LOG & HANDLE] → Log the operation and handle any exceptions with error messages.

Step 09 [LOOP / EXIT] → Return to Main Menu for next operation OR close DB and EXIT.

Step 10 [END] → Application terminates after secure database connection closure.